Sparse-conv Offline Perception with Hydra-MDP Inference: An Open Camera–LiDAR Planning Stack for NAVSIM

Atharv Sharma ATHARV228.AS@GMAIL.COM

Abstract

We release an open Apache-2.0 software stack for training, evaluating, and deploying an end-to-end autonomous driving planner on NAVSIM: *Sparse-conv Offline Perception with Hydra-MDP Inference*. A fused camera–LiDAR bird's-eye-view (BEV) feature $\mathbf{F}_{\text{env}}$ is shared by a Hydra-MDP-style trajectory decoder and auxiliary 3D detection and BEV segmentation heads. NVIDIA's CUDA-BEVFusion reference provides a high-performance detection runtime but ships a closed LiDAR sparse-convolution (SCN) binary and omits segmentation and planning. This release replaces that closed SCN path with open `spconv` 2.x, exports the full multi-head graph to ONNX/TensorRT, ships a one-frame `example-data/` sample for clone-and-run C++ inference, and documents a `.pth`→ONNX→TensorRT→C++ workflow. On an RTX 3060 we measure ∼38 ms LiDAR SCN latency (∼391 ms end-to-end C++ FP16). On the NAVSIM `navtest` split (12,149 scenarios) the released checkpoint scores PDM 0.7925 (NC 0.982, DAC 0.975, TTC 0.981). Python and C++ trajectories agree to within 1.7 mm max absolute error (cosine similarity 1.000).

Keywords: open source software, autonomous driving, bird's-eye view, sparse convolution, TensorRT, NAVSIM

1 Introduction

Learned planners such as Hydra-MDP Chen et al. (2024) and GTRS NVIDIA Corporation (2024) score a trajectory vocabulary from a dense BEV environment encoding. BEVFusion Liu et al. (2023) and NVIDIA's CUDA-BEVFusion NVIDIA Corporation (2022) supply a strong camera–LiDAR fusion template for detection, but the reference deployment (i) depends on a non-redistributable `libspconv` for the LiDAR SCN, (ii) exports detection only, and (iii) does not document a closed loop from NAVSIM Dauner et al. (2024) training through TensorRT planning.

This software fills that gap. We reuse the public BEVFusion backbone and a Hydra-style distillation decoder, and contribute: (1) an open SCN ONNX executor on `spconv` 2.x Yan (2022); (2) ONNX/TensorRT export and a C++ runtime for planning, detection, and BEV segmentation on a shared $\mathbf{F}_{\text{env}}$; (3) a one-frame `deploy/example-data/` sample so users can run C++ inference without downloading the full NAVSIM dataset; (4) validated navtest scores and Python↔C++ parity. The goal is that a researcher can clone the repository, convert a checkpoint, and redeploy without the closed SCN binary.

Related software. We build on BEVFusion Liu et al. (2023), lift-splat-shoot Philion and Fidler (2020), SECOND-style sparse ops Yan et al. (2018), open `spconv` Yan (2022), Hydra-MDP/GTRS Chen et al. (2024); NVIDIA Corporation (2024), and NAVSIM/PDM Dauner et al. (2024, 2023). Our contribution is the open integration and deployment path.

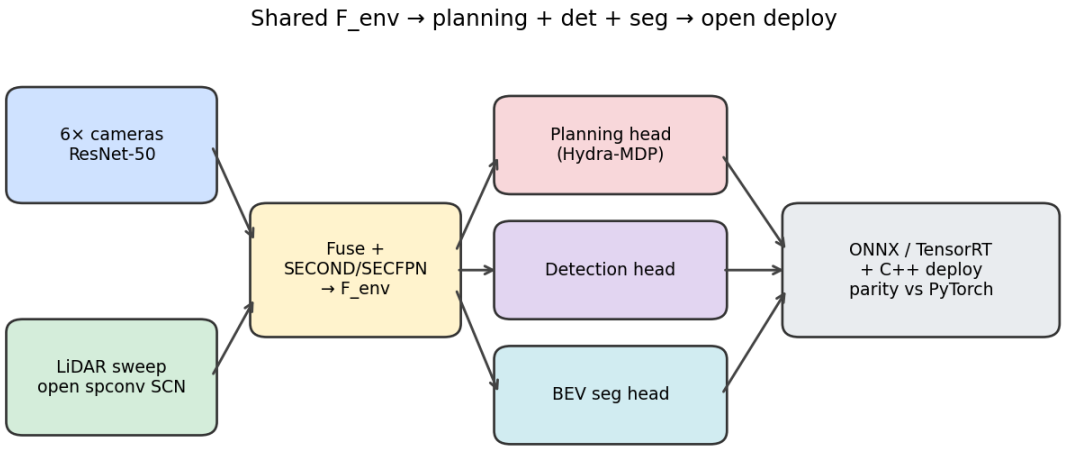

Figure 1: Pipeline layout. Camera (ResNet-50/LSS) and open-`spconv` LiDAR SCN fuse into $\mathbf{F}_{\text{env}}$, consumed by planning, detection, and BEV-segmentation heads. TensorRT runs dense graphs; SCN runs via the open parser.

2 Software Overview

Repository layout. The open-source release (Apache-2.0) is at `https://github.com/atharvsharma1998/hydra-mdp` (`main`): `navsim/agents/gtrs_bevfusion/`, `scripts/export/export_gtrs_be` `deploy/` (C++/TensorRT NVIDIA Corporation (2023) + `example-data/`), and `QUICKSTART.md` for the deploy path.

Architecture. Camera images and a LiDAR sweep produce $\mathbf{F}_{\text{env}} \in \mathbb{R}^{512\times H\times W}$ via LSS view transform Philion and Fidler (2020), voxel SCN, convolutional fusion, and a SECOND/SECFPN decoder (Figure 1). $\mathbf{F}_{\text{env}}$ is tokenized and fed to a Transformer trajectory scorer with a frozen vocabulary (up to K=8192), trained by soft imitation and per-proposal PDM sub-score distillation Dauner et al. (2023). Auxiliary detection and BEV segmentation heads supervise the same feature; they do not gate the plan.

Open SCN runtime. The LiDAR ONNX graph (∼53 nodes) is executed by `ONNXLiDARModel`: FP16 KRSC weights on GPU, topological `spconv` 2.x ops (`kMaskImplicitGemm` with tuner cache, SubM pair-table reuse, grow-only scratch buffers). This replaces NVIDIA's closed `libspconv`. Dense graphs are built with TensorRT 8.6 FP16 engines; the camera ResNet uses FP32 compute with FP16 I/O to avoid overflow.

3 User Workflow

Deploy (no full dataset required) follows `QUICKSTART.md`: (1) install CUDA/TensorRT/`spconv`/CUDA-BEVFusion core; (2) download the published `.pth` (or pre-exported ONNX); (3) export ONNX from the checkpoint when desired—tracing uses one mini-split frame via `--sensor-blobs-path` only to fix shapes, not the full benchmark; (4) build TensorRT engines with `build_gtrs_engines.sh`

`fp16`; (5) run `./build/gtrs_bevfusion example-data gtrs_bevfusion fp16` on the shipped one-frame sample. Training and full `navtest` PDM scoring are documented separately in `docs/TRAINING.md`.

4 Experimental Validation

Latency (RTX 3060 12 GB, FP16). Table 1 reports steady-state C++ timings (20-iteration mean).

Table 1: C++ FP16 module latency on RTX 3060 (ms, mean).

Module	ms
LiDAR (range-crop + open SCN)	37.8
Camera (LSS)	169.8
Fuser ($\mathbf{F}_{\text{env}}$)	1.6
Heads (plan + det + seg)	182.2
Total (device)	**391.3**

NAVSIM navtest PDM. On `navtest` (12,149 scenarios), checkpoint `gtrs_bevfusion_navtrain_v1_best.p` scores **PDM** 0.7925 (NC 0.982, DAC 0.975, TTC 0.981, EP 0.854, DDC 0.996, TLC 0.998, LK 0.970, comfort 0.964).

Python↔C++ parity. On token `8bc34517e08758ff`, cosine similarities are ≥ 0.993 (Table 2); trajectory $\max|\Delta|=1.7\times10^{-3}$ m.

Table 2: Python vs. C++/TensorRT parity (cosine / $\max|\Delta|$).

Stage	Cosine	$\max\|\Delta\|$
lidar_bev / cam_bev / $\mathbf{F}_{\text{env}}$	0.993 / 1.000 / 0.997	2.35 / 0.10 / 0.73
scores / trajectory	0.9995 / 1.000	1.57 / 0.0017
agent states / class logits	1.000 / 1.000	0.066 / 0.066
bev_semantic_logits	0.9998	1.27

5 Licensing and Limitations

Sources and documentation are Apache-2.0. Open `spconv` executes the LiDAR SCN. CUDA and TensorRT remain required for the documented C++ path. NAVSIM data is needed only for training and full evaluation; C++ smoke tests use shipped `example-data/`. Novelty is the open SCN path, multi-head deploy graph, and recipe—not a new planning algorithm. Future work includes INT8 TensorRT engines and ego-progress calibration.

6 Conclusion

We open-source a camera–LiDAR–planning stack for NAVSIM with open sparse-convolution inference, multi-head TensorRT deploy, a one-frame sample for clone-and-run C++ infer-

ence, and validated navtest PDM 0.7925 with millimeter-scale Python–C++ trajectory agreement.

Acknowledgments and Disclosure of Funding

No external funding was received. Source: `https://github.com/atharvsharma1998/hydra-mdp`.

References

Shoufa Chen, Shaoshuai Jiang, Hongyang Li, Yue Jia, Hongsheng Gao, Junchi Yan, and Hongyang Li. Hydra-MDP: End-to-end multimodal planning with hydra distillation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *Conference on Robot Learning (CoRL)*, 2023.

Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.

Zhijian Liu, Haotian Tang, Alexander Amini, Jing Yang, Hanjie Mao, Daniela Rus, and Song Han. Bevfusion: Multi-task multi-sensor fusion with unified bird's-eye view representation. In *International Conference on Robotics and Automation (ICRA)*, 2023.

NVIDIA Corporation. CUDA-BEVFusion: High-performance 3D perception for autonomous driving. `https://github.com/NVIDIA-AI-IOT/Lidar_AI_Solution`, 2022. Accessed 2026-06-13.

NVIDIA Corporation. TensorRT: High-performance deep learning inference. `https://developer.nvidia.com/tensorrt`, 2023.

NVIDIA Corporation. GTRS: Generalized trajectory scoring for autonomous driving. `https://github.com/NVIDIA/DRIVE-Lab/GTRS`, 2024. Accessed 2026-06-13.

Jonah Philion and Sanja Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3D. In *European Conference on Computer Vision (ECCV)*, 2020.

Yan Yan. Spconv: Spatially sparse convolution library. *GitHub repository*, 2022. `https://github.com/traveller59/spconv`.

Yan Yan, Yuxin Mao, and Bo Li. SECOND: Sparsely embedded convolutional detection. In *Sensors*, 2018.